

Seismic-Sim

The Math, Explained From Scratch

This document assumes **no prior background** — not in structural engineering, not in the kind of math (matrices, eigenvalues, Fourier transforms) used to solve the problem. Every idea is introduced with a plain-English explanation and an analogy before any equation shows up. If you read start to finish, you should come away understanding not just *what* `mdof_response.py` computes, but *why* each step exists.

Companion to `README.md` — that document covers what each file in the project does; this one is about the ideas and numbers inside `mdof_response.py` specifically. Keep both updated as the project grows.

Part A builds up every concept you need from zero. **Part B** then walks through the actual pipeline, reusing those concepts by name. If a term ever feels unfamiliar while reading Part B, it was covered in Part A — jump back.

Part A — Foundations

Five ideas, explained once, that everything later depends on.

A1. The big picture

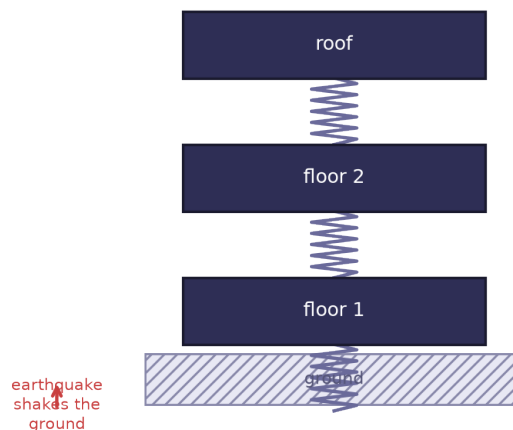
Picture a real multi-story building. When the ground shakes, the building's base is forced to move with it — but the upper floors don't move in perfect lockstep. Each floor is connected to the ones above and below it by the building's structure (columns, walls), which behaves a little like a spring: it resists being stretched or compressed, but isn't perfectly rigid.

Think of it like this: stack a few mattresses on top of each other and shove the bottom one sideways. The bottom mattress moves first; the ones above lag behind, then overshoot, compressing and stretching against their neighbors as the whole stack sways. A building does the same thing, just stiffer and faster.

This project replaces the real building with the simplest model that captures that behavior: a stack of flat, heavy “floors,” each connected to its neighbor by a spring. Every floor can move sideways somewhat independently of the others. The goal of all the math that follows is: given how the ground moves, figure out exactly how far *each floor* moves, at *every instant*.

What `mdof_response.py` builds:
a free-top cantilever

nothing above the roof —
it's genuinely free to sway



The building model: floors (dark boxes) stacked on springs (zig-zag lines), sitting on the shaking ground (hatched). Fixed at the base, genuinely free at the roof — a real free-standing cantilever, not a chain propped up from above too. More in Part B, §B2.3.

A2. Describing many floors at once: lists of numbers

If a building has 3 floors, and at some instant floor 1 has moved 2 mm sideways, floor 2 has moved 5 mm, and floor 3 has moved 4 mm, the most natural way to write that down is just an ordered list:

2	millimeters (floor 1, floor 2, floor 3, in order)
5	
4	

In math, an ordered list of numbers like this is called a **vector**. Nothing more exotic than that — it's just a column of numbers where *position in the list* means something (here: which floor). The code stores floor positions this way constantly.

A3. Describing relationships between floors: grids of numbers

A vector can say how far each floor has moved. But to *calculate* the motion, we also need to describe two physical facts about the building: how heavy each floor is, and how the springs connect neighboring floors. Both are naturally described as a grid of numbers — a **matrix** — with one row and one column per floor.

The mass matrix M — “how heavy is each floor?”

For our 3-floor example (each floor weighing 1,000,000 kg — 1,000 tonnes, a plausible number for a real floor plate), the mass matrix looks like this:

M =	1,000,000	0	0	kilograms (kg) 1,000,000 kg = 1,000 tonnes, a plausible weight for one floor
	0	1,000,000	0	
	0	0	1,000,000	

Every floor's own weight sits on the diagonal (top-left to bottom-right); everywhere else is zero, because in this simple model, one floor's weight has nothing directly to do with another floor's weight. A matrix that's zero everywhere except the diagonal is one of the simplest kinds there is.

The stiffness matrix K — “how do the springs pull on each other?”

The springs are more interesting, because each one connects *two* floors — pulling on both of them whenever they're stretched or compressed relative to each other. For our 3-floor example, with a spring stiffness of about 2215 million newtons per meter (a stiffness value, in the code called *k*, chosen so the building sways at a realistic speed — see §B2):

$$K = \begin{bmatrix} 4,429.4 & -2,214.7 & 0 \\ -2,214.7 & 4,429.4 & -2,214.7 \\ 0 & -2,214.7 & 2,214.7 \end{bmatrix}$$

meganewtons per meter (MN/m). 1 MN is roughly the weight-force of 100 cars.

Read row 2 (the middle row) as: “the force pulling floor 2 back toward its resting position depends on floor 2’s own position (the middle number, positive — pulling it back) and on floors 1 and 3’s positions too (the outer numbers, negative — a neighbor moving away drags floor 2 along with it).” Every row tells the same kind of story for its own floor. That’s all a stiffness matrix is: a compact table of “who pulls on whom, and how hard.”

Note: Look closely at the last row (floor 3, the roof): its diagonal entry is *half* the others, and there’s no entry pulling it from “above” — because there’s nothing above the roof to pull it back. Every other floor has a neighbor both below and above it, so it gets $2k$ on the diagonal; the roof only has a neighbor below, so it gets just k . This is what makes the model a genuine free-standing cantilever rather than a chain propped up at both ends — see §B2.3.

A4. A building's favorite ways to wobble: natural frequencies and mode shapes

Push a swing and let go — it doesn’t swing at just any speed you like; it settles into one particular back-and-forth rhythm determined by the swing’s length. Pluck a guitar string and it doesn’t vibrate randomly either — it vibrates in a small number of specific shapes (the fundamental note, then higher harmonics), each at its own specific pitch.

Think of it like this: a stack of spring-connected floors behaves the same way. Left alone to vibrate, it doesn’t wobble chaotically — it has a small, fixed number of **natural sway shapes** (one per floor), each with its own natural speed. Shake the building at exactly one of those speeds and it responds strongly in exactly that shape (resonance) — just like singing the right note can make a wine glass ring.

Each natural sway shape is called a **mode shape** (usually written with the Greek letter ϕ , phi) — literally just a vector, like in §A2, saying how far each floor moves *relative to the others* when the building sways in that particular pattern. Each mode shape has a matching **natural frequency** (written ω , omega) — how fast that particular pattern oscillates back and forth.

Finding them: the eigenvalue problem

Finding these natural shapes and speeds means finding the vectors ϕ and numbers ω that satisfy one condition: when the building is vibrating in shape ϕ , the spring forces trying to pull it back ($K\phi$, using the stiffness matrix from §A3) must be exactly proportional to how hard inertia resists that motion ($\omega^2 M\phi$, using the mass matrix). In other words: a shape where the whole system’s forces stay in step with each other as it oscillates, neither building up energy nor losing it, so it can self-sustain.

$$K\phi = \omega^2 M\phi$$

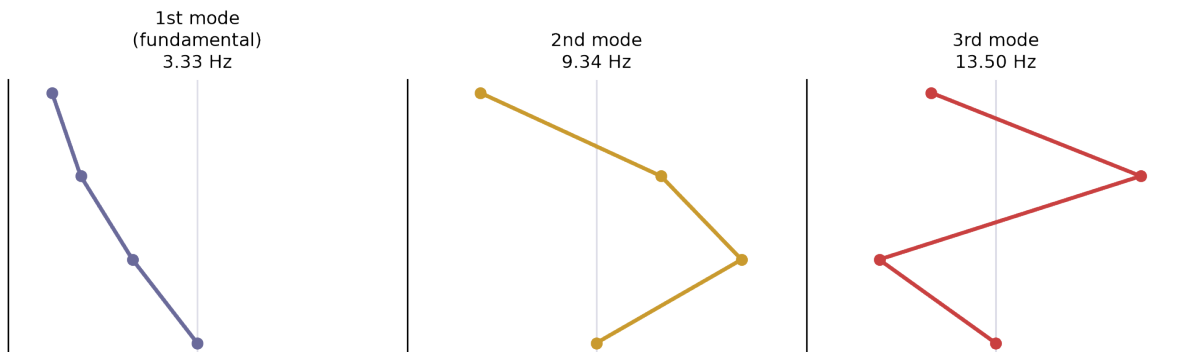
This kind of equation — find the special vectors and numbers that make a matrix equation like this balance — is called an **eigenvalue problem** in math (“eigen” is German for “own”/“characteristic” — literally, “the system's own, characteristic values”). ω^2 is the **eigenvalue**, ϕ is the matching **eigenvector**. This is a well-known, solved problem in math — the code hands it to a library function (`scipy.linalg.eigh`) rather than solving it by hand.

For the 3-floor example from §A3, solving this gives 3 natural frequencies and 3 matching mode shapes:

$$f_1 = 3.33 \text{ Hz}, \quad f_2 = 9.34 \text{ Hz}, \quad f_3 = 13.50 \text{ Hz}$$

Hz = cycles per second. Higher modes always oscillate faster than lower ones.

The 3 natural sway shapes of the N=3 example building
(fundamental mode grows toward the free roof, not a symmetric hump)

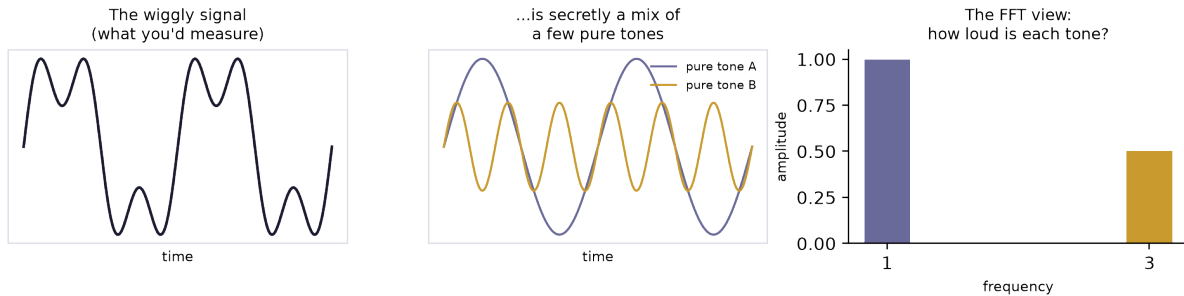


The 3 natural sway shapes for the example building. The fundamental mode (left) grows toward the free roof rather than forming a symmetric hump — a direct consequence of the free-top cantilever topology from §A3.

A real earthquake doesn't shake a building in one clean mode shape — the actual motion is always some mix of all the modes at once, each contributing more or less depending on how strongly the ground motion “matches” that mode's natural rhythm. The code computes each mode's response separately (§B3) and adds them back together (§B3.4) to get the real answer — this is exactly the same trick as unmixing a chord into its individual notes, and it's why §A5 matters next.

A5. Unmixing a signal into pure tones: the Fourier transform

Any sound you hear — a chord, a voice, noise — can be broken down into a sum of simple, pure tones (single-frequency sine waves) played together at different volumes. Your ear (and a piano tuner, and an equalizer app) does this unmixing constantly. The mathematical version of this unmixing is called a **Fourier transform**; the fast computer algorithm that performs it is the **FFT (Fast Fourier Transform)**.



The same signal, seen two ways: as a wiggly line over time (left/middle), or as a list of “how much of each pure tone is present” (right). The FFT converts between these two views.

Two names for the same signal, from two different angles:

- **Time domain** — the signal as most people picture it: value vs. time. A ground-motion recording, before any processing, is in the time domain.
- **Frequency domain** — the same signal described instead as “how much of each frequency it contains.” Converting to this view is what the FFT does.

Why bother converting? Because some calculations that are painful in the time domain become simple multiplication in the frequency domain. Two examples used directly in this project:

- **Integration/differentiation** (going from acceleration to displacement, or back — needed when a recording only gives you one of the two, §B2) becomes just dividing or multiplying by the frequency.
- **“How does mode i respond to this ground motion?”** (§B3) becomes multiplying the ground motion's frequency-domain view by a per-mode “response recipe” — instead of simulating the mode's motion instant-by-instant.

Whenever you see a hat over a symbol below, like $\hat{a}(\omega)$, it means “this signal, viewed in the frequency domain” (as opposed to plain $a(t)$, the same signal in the time domain). ω (omega) is used for frequency here, measured in radians per second rather than Hz — just a different, equally valid way of counting “how fast.”

A6. Why swaying dies down: damping

Push a swing once and, left alone, it doesn't swing forever — friction and air resistance gradually steal its energy until it stops. Buildings have an equivalent effect: internal friction in the structure, gradually converting sway motion into heat, without which any shaking would (unrealistically) continue forever. This effect is called **damping**, written with the Greek letter ζ (zeta), usually expressed as a percentage. The code uses a fixed 5% (zeta = 0.05) for every mode — a standard, generic assumption for a typical building, not measured from a specific real structure.

Note: 5% damping is a common textbook default for buildings, not a universal constant — real damping varies by construction type, materials, and even shaking intensity. If you ever want a more building-specific simulation, this is one of the first numbers worth revisiting.

Part B — The Pipeline in `mdof_response.py`

With Part A's vocabulary in hand, here is every calculation the code actually performs, in order.

B1. Reading a clean ground-motion signal

B1.1 Units

PEER acceleration files are usually measured in multiples of Earth's gravity, g (9.80665 m/s² — the constant acceleration anything in free-fall near Earth's surface experiences). The code converts to standard metric units:

$$a_{[\text{m/s}^2]} = a_{[g]} \times 9.80665$$

Displacement files (.DT2) are typically in centimeters and are divided by 100 to get meters.

B1.2 Recovering a missing acceleration or displacement record

The model needs two related but different pieces of information about the ground: how fast it's *accelerating* (this is what actually pushes the building — the “forcing” input) and how far it has physically *moved* (needed to know the building's true position in space, not just its sway relative to the ground). When a recording only supplies one of the two, the code derives the other using the frequency-domain trick from §A5: repeated multiplication or division by the frequency (ω) is mathematically the same as differentiating or integrating, but far cheaper and more stable to compute than doing it directly on the wiggly time-domain signal.

$$\hat{d}(\omega) = \frac{-\hat{a}(\omega)}{\omega^2} \quad \implies \quad d(t) = \mathcal{F}^{-1}\{\hat{d}(\omega)\}$$

Double integration: acceleration -> displacement

$$\hat{a}(\omega) = -\omega^2 \hat{d}(\omega) \quad \Rightarrow \quad a(t) = \mathcal{F}^{-1}\{\hat{a}(\omega)\}$$

Double differentiation: displacement -> acceleration

(The $\mathcal{F}^{-1}$ symbol just means “convert back from the frequency domain to the time domain” — the reverse of the FFT from §A5.)

Note: Dividing by ω^2 blows up as ω approaches 0, and real recordings always carry a little unwanted very-low-frequency drift/noise. Before dividing, the code passes the signal through a smooth **high-pass filter** (a 4th-order Butterworth filter, applied with zero phase distortion) instead of bluntly zeroing out FFT bins — avoids the ringing a sharp on/off cutoff can introduce. The cutoff itself is adaptive: capped at 0.1 Hz (integration) / 0.05 Hz (differentiation), but never set higher than a fraction of the building's own fundamental frequency, so a very soft, long-period building's genuine low-frequency structural response doesn't get filtered out along with real drift. This is the standard “baseline correction” trick used throughout earthquake engineering to stop integrated displacement from drifting off to infinity.

B2. Building the MDOF shear model

“MDOF” = multi-degree-of-freedom, from the glossary in README.md — a model with several parts (here, floors) that can each move somewhat independently. “Shear building” is the specific simplified version from §A1: rigid floors, all the flexibility concentrated in springs between them.

B2.1 Mass matrix

As in §A3: every floor gets the same weight, so the mass matrix M is simply that weight repeated down the diagonal:

$$M = m \cdot I_N$$

(I_N means “the identity matrix” — 1s down the diagonal, 0s elsewhere; multiplying it by m just puts the floor mass on every diagonal entry, exactly like the worked example in §A3.)

B2.2 Target frequency → spring stiffness

Rather than picking a spring stiffness out of thin air, the model works backward from a realistic *target*: real buildings tend to have a fundamental sway period (the slowest, biggest natural mode from §A4) of roughly 0.1 seconds per floor — a well-known engineering rule of thumb. For an N -story building, that target period is $T1_factor \times N$ seconds, converted to a frequency:

$$\omega_1 = \frac{2\pi}{T_1}$$

The per-floor spring stiffness k is then solved for using a known formula for the fundamental frequency of a uniform chain of equal masses and springs that's anchored (fixed) at the base and genuinely free at the top — matching the diagram back in §A1:

$$k = m \left(\frac{\omega_1}{2 \sin\left(\frac{\pi}{2(2N + 1)}\right)} \right)^2$$

For our 3-floor, 1,000-tonne-per-floor example, targeting a 0.3-second fundamental period, this works out to about 2215 million newtons per meter per spring — the exact value used to build the matrix shown back in §A3.

B2.3 Stiffness matrix

K is assembled to match the free-top cantilever topology from §A1: every floor connects downward to its neighbor with $-k$. Floors below the roof also have a neighbor above them, so they connect upward with $-k$ too and carry $2k$ on their own diagonal. The roof only has a neighbor below — nothing pulls it

back from above, because there's nothing there — so its diagonal is just k . Exactly the pattern filled in with real numbers in the worked example of §A3.

$$K_{ii} = 2k \ (i < N), \quad K_{NN} = k, \quad K_{i,i-1} = K_{i,i+1} = -k$$

Note: This one-entry difference (k instead of $2k$ on just the last diagonal entry) is the entire structural difference between a real free-standing building and a chain propped up at both ends. It's a small-looking change in the matrix with a real consequence: it's what makes the fundamental mode shape grow toward the roof (see §A4's diagram) instead of forming a symmetric hump.

B2.4 Modal analysis

This is exactly the eigenvalue problem from §A4, solved with the real M and K matrices above, using `scipy.linalg.eigh(K, M)`:

$$K\phi = \omega^2 M\phi$$

The results are sorted from lowest to highest frequency, then each mode shape is rescaled (“mass-normalized”) to a standard, consistent size, so the modes can be fairly compared and combined later:

$$\phi_i^T M \phi_i = 1$$

Finally, each mode gets a **participation factor** Γ (Gamma) — a single number capturing how strongly a uniform push on the whole building's base (like an earthquake) excites that particular mode. A mode shape that happens to be symmetric about the building's midpoint would get a near-zero participation factor, since a uniform push can't tell its two “halves” apart — but a real free-standing cantilever's mode shapes (see §A4's diagram) aren't symmetric like that, so every mode below picks up a genuine, nonzero contribution:

$$\Gamma_i = \phi_i^T M \mathbf{1}$$

(The bold $\mathbf{1}$ just means “a vector of all 1s” — representing “every floor pushed the same amount.”) For the 3-floor example, the participation factors work out to $\Gamma_1 \approx -1656$, $\Gamma_2 \approx 474$, $\Gamma_3 \approx -182$ — all three meaningfully nonzero. (An earlier version of this model was built on a different, symmetric topology where one mode was barely excited by construction — a quirk of that symmetry, not a real structural feature; the current free-top cantilever doesn't have that quirk.)

B3. Solving the response in the frequency domain

B3.1 Each mode behaves like its own simple pendulum

Here's the payoff of all the setup above: instead of solving the motion of all N connected floors together (hard — they all affect each other), the modal analysis in §B2.4 lets us treat each mode as if it were its own *completely independent*, simple back-and-forth oscillator (like a single pendulum, or a single mass on a single spring) — no interaction with the other modes at all. Mode i 's own “position” over time (called its **modal coordinate**, $q_i(t)$) obeys the standard equation for a damped oscillator being pushed by an outside force — here, the ground acceleration, scaled by that mode's participation factor from §B2.4:

$$\ddot{q}_i + 2\zeta\omega_{n,i}\dot{q}_i + \omega_{n,i}^2 q_i = -\Gamma_i a_g(t)$$

(The dots over q are a standard shorthand from physics for “rate of change” — one dot = velocity, two dots = acceleration. You don't need to solve this by hand; the next step shows how the code solves it.)

B3.2 Solving instantly with a frequency-response recipe

Because each mode behaves like a simple, well-understood oscillator (§B3.1), there's a known shortcut: instead of stepping the equation forward moment-by-moment, you can compute exactly how that mode would respond to a single pure tone at any given frequency — a “recipe” function $H_i(\omega)$. This is precisely the “mix of pure tones” idea from §A5:

$$H_i(\omega) = \frac{-\Gamma_i}{\omega_{n,i}^2 - \omega^2 + i2\zeta\omega_{n,i}\omega}$$

Then, because the FFT (§A5) already tells us exactly which pure tones the real ground motion is made of and how loud each one is, the mode's full response is just: for every pure tone in the ground motion, multiply by how that mode reacts to a tone at that particular frequency, then add all the (now individually solved) tones back together:

$$\hat{Q}_i(\omega) = H_i(\omega) \hat{A}(\omega) \quad \Rightarrow \quad q_i(t) = \mathcal{F}^{-1}\{\hat{Q}_i(\omega)\}$$

Think of it like this: this is the same shortcut as knowing exactly how a wine glass rings when hit with any single pure note, then figuring out how it responds to a whole noisy room by breaking that noise into notes, looking up the glass's response to each one individually, and adding the results — instead of trying to model the glass's physical vibration through the whole noisy room directly.

This is done once per mode, for the entire time history at once — no step-by-step time-marching simulation (a common alternative method called Newmark- β , not used anywhere in this codebase) is

needed.

B3.3 Avoiding a subtle bug: zero-padding before the FFT

There's one more wrinkle worth knowing about. `fft/iff` on a finite-length recording doesn't quite compute “how would this mode respond forever” — it computes a *circular* convolution, which secretly treats the recording as if it looped back to its own start the instant it ends. A lightly-damped mode's response to an early jolt can still be ringing by the end of the recording, and in a circular convolution that ringing wraps around and contaminates the very start of the computed response.

Think of it like this: imagine a bell still faintly ringing when a song loops back to bar one — in a circular world, that fading ring gets glued onto the beginning of the next play-through, even though nothing actually rang there.

The fix is standard: pad the recording with extra zeros before the FFT (giving the ringing somewhere to decay into instead of wrapping around), then trim the padding back off after inverse-transforming. How much padding is enough depends on how long the slowest, least-damped mode takes to die down — about 5 time constants, $1/(\zeta\omega_n)$ each, for that mode:

$$\text{pad_len} = \min \left(4n, \quad n + \left\lceil \frac{5}{\zeta\omega_1 dt} \right\rceil \right)$$

n = original recording length. Capped at 4x so an extremely soft, lightly-damped building can't make a single computation take unbounded time.

B3.4 Adding the modes back together

Every mode has now been solved on its own (§B3.2). Real floor motion is just all of them added back together, each weighted by how much that floor moves within that mode's particular shape (§A4):

$$u_{\text{rel}}(t) = \sum_{i=1}^N \phi_i q_i(t)$$

This gives each floor's motion *relative to the ground* — how much the building is flexing, ignoring the fact that the ground itself is also physically moving through space. Adding the ground's own motion back on top gives each floor's true, absolute position:

$$u_{\text{abs}}(t) = u_{\text{rel}}(t) + u_g(t) \mathbf{1}$$

$u_{\text{abs}}(t)$ — absolute floor position over time — is the final answer this whole pipeline exists to compute. It's what gets saved to `response_X.csv` / `response_Y.csv`.

B4. From spreadsheet to animated sway

For the record you first pick, and during playback, nothing in the browser re-derives any of this physics — index.html only plays back the $u_{\text{abs}}(t)$ values already computed above and saved to the CSV files. (The one exception, live recompute via the Building Parameters sliders, is covered separately in §B5.) Per rendered frame, each floor's on-screen position is simply:

$$\text{position}_{\text{floor}}(t) = \text{amplify} \times u_{\text{abs},\text{floor}}(t)$$

amplify is the UI slider from README.md's glossary. Real inter-floor sway can be anywhere from a fraction of a millimeter to tens of centimeters depending on the earthquake — far too small to see clearly on screen either way — so amplify's default is auto-computed per record, and its range is logarithmic, so it stays useful across that whole spread. It scales the picture up purely for visibility; it never changes the underlying numbers. Playback advances through the same time steps the physics was computed at, at a rate controlled by a separate, independent playback-speed slider.

B5. Live recompute: the same math, run again on demand

The Building Parameters sliders (stories, mass per floor, damping, target period) are the one place actual physics runs outside the original offline pipeline. Moving one sends those four numbers, plus which earthquake record is selected, to a small backend (server.py), which does exactly what mdof_response.py already does — build a fresh MDOF_ShearBuilding (§B2), solve its modes (§B2.4), and run the same FFT-based response calculation (§B3) — just re-invoked live, with your chosen numbers instead of the fixed defaults out/ was generated with. No new math: it is the same M , K , and $H_i(\omega)$ from earlier in this document, computed again on demand.

What does *not* get recomputed: the ground motion itself ($a_g(t)$ and $u_g(t)$) — that only depends on the earthquake recording, never on building parameters, so it is parsed and baseline-corrected once per record (§B1) and cached, not redone on every slider move.

Note: The physics itself is fast — tens of milliseconds, since N is small and the FFT solve is already vectorized. What actually took work to get fast was *transmitting* the result: sending the full recomputed time series as JSON text on every slider move measured out taking longer than the physics itself by more than an order of magnitude, purely from formatting/parsing hundreds of thousands of numbers as decimal text. The fix was a compact binary response (raw float32 arrays) instead of JSON for the large arrays — an engineering detail, not a physics one, but the reason the sliders feel live rather than sluggish.

Symbol reference

Symbol	Plain-English meaning	Where in code
N	Number of floors being simulated	self.N
m	Weight of one floor (kg)	self.m, mass_per_floor
k	How stiff each connecting spring is (N/m)	self.k
M, K	The mass / stiffness grids of numbers (§A3)	self.M, self.K
ζ (zeta)	Damping — how fast sway dies out (§A6)	self.zeta
ω_n (omega_n)	A mode's natural oscillation speed (§A4)	self.omega_n
ϕ (phi)	A mode's natural sway shape (§A4)	self.phi
Γ (Gamma)	How strongly a mode gets excited (§B2.4)	self.Gamma
a_g(t)	Ground acceleration over time (the input)	acceleration / self.accel
u_g(t)	Ground displacement over time	displacement / self.ground_disp
q_i(t)	Mode i's own simple back-and-forth motion	q_time[i, :]
u_rel(t)	Floor motion relative to the ground (flex only)	self.floor_disp_rel
u_abs(t)	Floor's true position in space (the final answer)	self.floor_disp_abs
dt	Time between each data point in the recording (s)	dt

Source: *mdof_response.py* (*MDOF_ShearBuilding* class). See *README.md* for how this fits into the rest of the codebase.